Project Report — Machine Learning

Reinforcement Learning in the Iterated Prisoner's Dilemma:

Convergence and Comparative Analysis of Strategies

Gabriel Tseng Axel Rubio Benjamin Krief
Krea Fares

Master 203 — Université Paris-Dauphine

Machine Learning Course — March 23, 2026

Abstract

Game theory provides a rigorous framework for analysing strategic interactions in which individual incentives diverge from collective welfare, a tension that arises naturally in multi-agent systems like financial markets. The Iterated Prisoner's Dilemma (IPD) is a canonical instance of this tension, and applying reinforcement learning to it allows us to study how adaptive agents learn to navigate without any prior knowledge of the game structure.

This report will go through the application of Q-learning to the IPD. There are two objectives: first, to understand how a rational learning agent behaves when facing classical deterministic strategies; second, to assess the relative importance of Q-learning hyperparameters : learning rate α, discount factor γ, ε-greedy exploration policy.

Our experiments show that a Q-learning agent converges to systematic defection when facing any player that do not punish defection, consistent with the dominant Nash equilibrium of the one-shot game. We compare its performance against a set of baseline strategies, ranging from naive heuristics (*Always Cooperate*, *Always Defect*) to reactive strategies (*Tit for Tat* and its variants), and discuss the conditions under which cooperation can or cannot emerge in a mixed population.

Contents

1 Introduction: The Prisoner's Dilemma **3**
1.1 Game description . . . 3
1.2 The iterated version (IPD) . . . 3
1.3 Research question . . . 4
1.4 Outline . . . 4

2 Experimental Framework and Strategy Set **5**
2.1 Simulator architecture . . . 5
2.2 Implemented strategies . . . 5
2.2.1 Naive strategies (baseline heuristics) . . . 5
2.2.2 Reactive strategies . . . 5

3 The Q-Learning Model **6**
3.1 Justification of choice . . . 6
3.2 Mathematical formulation . . . 6

4 Comparison with Baseline Strategies **7**
4.1 Evaluation framework . . . 7
4.2 Comparative results . . . 7
4.2.1 Naive strategies . . . 7
4.2.2 Reactive strategies . . . 8

5 Hyperparameter Sensitivity Analysis **9**
5.1 Impact of ε, exploration parameter . . . 9
5.2 Impact of α, learning rate . . . 10
5.3 Impact of γ, discount factor . . . 11

6 Experimental Results **11**

7 Conclusion **11**

1 Introduction: The Prisoner's Dilemma

1.1 Game description

The Prisoner's Dilemma is a symmetric two-player game originally formulated by Tucker [1950] and popularised in the context of game theory by Axelrod [1984]. At each round, two players simultaneously and independently choose between two actions: **cooperate** (C) or **betray** (B). The payoff matrix is defined as follows:

Table 1: Payoff matrix of the Prisoner's Dilemma (row = player 1, column = player 2).

	Cooperate (C)	**Betray (B)**
Cooperate (C)	(3, 3)	(0, 4)
Betray (B)	(4, 0)	(1, 1)

Those payoffs are defined such that they satisfy the following inequalities (as long as they do, the game is a Prisoner's Dilemma):

$$T > R > P > S \tag{1}$$

where

- T is the temptation to defect (4)
- R is the reward for mutual cooperation (3)
- P is the punishment for mutual defection (1)
- S is the sucker's payoff (0)

The structure of this game gives rise to a fundamental tension between individual rationality and collective optimum: **betraying is a strictly dominant strategy** as it yields a higher payoff regardless of the other player's action, yet if both players choose it, the outcome $(B, B) = (1, 1)$ is Pareto-dominated by $(C, C) = (3, 3)$.

1.2 The iterated version (IPD)

In the Iterated Prisoner's Dilemma (IPD), both players interact a large number of times. In the original version, they played 200 times. Here, we set $T = 500{,}000$ rounds for convergence purposes. The repetition of the game changes the analysis: the possibility of punishing past defection and rewarding past cooperation allows conditional strategies to establish themselves. Axelrod demonstrated empirically, through computer tournaments, that the *Tit for Tat* strategy (cooperate on the first move, then mirror the opponent's previous action) achieves remarkable performance in an ecosystem of varied strategies. In fact, the strategy won in the two tournaments he organized.

1.3 Research question

This project will try to answer to the following question:

Can an agent equipped with reinforcement learning (Q-learning) learn an effective strategy in the IPD, and under what conditions does it converge to cooperation or defection?

1.4 Outline

In order to answer this question, we will first implement a Q-learning agent from scratch and confront it with a controlled set of opponents in a battle. Not taking into account any prior knowledge of the game, we observe and measure the agent's emergent behaviour : its cooperation rate, its exploitation of weak opponents, its response to punitive strategies, and compare these outcomes to the Nash equilibrium prediction. An analysis over the key hyperparameters of the model (α, γ, ε) then allows us to highlight the role of each component of the learning rule. We shall then extend the analysis to multiple Q-learning agents interacting with each other, and observe the emergence of cooperation or defection in a mixed population.

The report is organised as follows. Section 2 presents the full set of implemented strategies and the experimental framework. Section 3 describes the Q-learning model, its mathematical formulation, and the rationale for this choice. Section 4 compares baseline strategy performance. Section 5 analyses hyperparameter sensitivity. Section 6 presents our experimental results, and Section 7 concludes.

2 Experimental Framework and Strategy Set

2.1 Simulator architecture

The simulator is built around three components: a `Strategy` (the decision rule), an `Agent` (a player carrying a strategy, a score, and a local interaction memory), and a `Game` (the orchestrator). At each round, both agents simultaneously submit an action to the `Game`, which resolves payoffs from the matrix (same as in 1), updates scores, and writes the outcome to each agent's memory. All distinct agent pairs are played exhaustively for $T = 500{,}000$ rounds.

2.2 Implemented strategies

In order to test a learning agent, we implemented a set of strategies to play against, drawn from the canonical taxonomy of IPD strategies [Kuhn, 2019]. These strategies can be divided into two categories: naive strategies and reactive strategies.

2.2.1 Naive strategies (baseline heuristics)

`AlwaysCooperate` Cooperates unconditionally at every round. Represents the maximum cooperation bound.

`AlwaysBetray` Defects systematically. Constitutes the Nash equilibrium of the IPD.

`RandomAction` Defects or cooperates with the same probability $\frac{1}{2}$.

`ProbaCooperation` Cooperates with a fixed probability p (parameter).

2.2.2 Reactive strategies

`TitForTat` (TFT) Cooperates on the first round, then mirrors the opponent's previous action.

`SuspiciousTitForTat` Identical to TFT but starts with a defection.

`TitForTwoTats` Only defects after two consecutive defections by the opponent (more lenient).

`TwoTitsForTat` Defects twice following a single received defection (more punitive).

`Joss` TFT with noise: defects randomly with probability p_j to exploit cooperators.

`DiscriminatingAltruist` Cooperates with any player who has never defected; otherwise refuses engagement.

`Bully` Defects until the opponent defects, then cooperates. Exploits accommodating strategies.

3 The Q-Learning Model

3.1 Justification of choice

We wanted to analyse the Prisoner's Dilemma from a different perspective than the one of game theory. As part of the game theory, this obvioulsy a game. As such, an agent at each state of the game will earn a reward. From a machine learning perspective, this is a reinforcement learning problem. Therefore, we could chose between different RL algorithms (SARSA, Deep Q-Network etc.). We chose Q-learning because it is a simple and efficient algorithm that can be used to solve this problem.

Indeed, SARSA is an on-policy algorithm, which onverges to a different policy under continuous exploration; it is less suitable here since we wish to evaluate the optimal policy. On the other hand, Deep Q-Network is a more complex algorithm that requires more data to train. Here, the space is minimal, so we don't need to use a neural network to approximate the Q-values.

Regarding the IDP and the Q-learning algorithm, we make the following assumptions:

- **Suitability for discrete spaces**: the IPD has a finite, discrete state space (the memory is reduced to each player's last action) making the tabular representation (q-table) efficient.
- **Off-policy**: the agent can learn the optimal policy even while exploring, which is relevant for the IPD where exploration allows testing the opponent's reactions.
- **Guaranteed convergence** under standard conditions.

3.2 Mathematical formulation

State space. The state s_t at time t is the pair of actions from the previous round.

Action space. $\mathcal{A} = \{C, B\}$.

Reward. The immediate reward r_t is the payoff matrix value obtained at round t (cf. Table 1).

Update rule (Bellman equation). The Q-table is updated after each round according to:

$$Q(s_t, a_t) \;\leftarrow\; Q(s_t, a_t) \;+\; \alpha\Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big], \tag{2}$$

where $\alpha \in (0, 1]$ is the learning rate and $\gamma \in [0, 1)$ the discount factor (valuation of future rewards).

ε-greedy policy. The agent selects a random action with probability ε (exploration) and the action with the highest Q-value with probability $1 - \varepsilon$ (exploitation). ε decays multiplicatively at each step by a factor `epsilon_decay`, down to a floor `epsilon_min`. We could use a constant epsilon, but the goal is to learn the optimal policy, so we need to explore at the beginning and then exploit.

4 Comparison with Baseline Strategies

4.1 Evaluation framework

Strategies are evaluated on several metrics computed at the end of the simulation:

- **Cooperation rate**: proportion of C moves played by the agent;
- **Exploitation rate** (defecting when the opponent cooperates);
- **Victimisation rate** (cooperating when the opponent defects);
- **Mutual cooperation rate**: proportion of (C, C) rounds.

4.2 Comparative results

For the first presented results, we chose the following set of parameters (the most sensitive one being gamma, our agent here values future rewards a lot):

α	γ	ε_0	ε_{min}	$\varepsilon_{\text{decay}}$
0.1	0.9	1.0	0.01	0.9999

4.2.1 Naive strategies

Table 2: Behavioural summary of the Q-learning agent against naive strategies.

Strategy	Coop.	Exploit.	Victimised	Mut.C
AlwaysCooperate	1.4%	98.6%	0%	1.4%
AlwaysBetray	1.4%	0%	1.4%	0%
RandomAction	2.6%	48.7%	1.3%	1.3%
ProbaCooperation(p=0.7)	2.5%	68.2%	0.8%	1.8%

These results are fully consistent with game-theoretic predictions: the agent converges to systematic defection in all cases.

Against `AlwaysCooperate`, exploitation reaches 98.6%; the residual 1.4% cooperation is due to the exploration phase, not a learned behaviour : the agent may cooperate at the beginning of the game but then exploits the opponent. Against `AlwaysBetray`, the agent reciprocates by defecting as well, converging to the Nash equilibrium (B, B).

Notably, the exploitation rate is closely related to the opponent's cooperation probability: $\approx 48.7\%$ against `RandomAction` ($p = 0.5$) and $\approx 68.2\%$ against `ProbaCooperation` ($p = 0.7$), confirming that the agent has learned to consistently defect.

The conclusion of this first part is that cooperation cannot arise againt naive strategies : **defection must be punished and cooperation must be rewarded**.

4.2.2 Reactive strategies

As we have seen in the previous section, cooperation cannot arise against naive strategies. However, it may arise against reactive strategies. This section shows how our agent behaves against reactive strategies.

Table 3: Behavioural summary of the Q-learning agent against reactive strategies.

Strategy	Coop.	Exploit.	Victimised	Mut.C
TitForTat	98.6%	1.2%	1.2%	97.4%
SuspiciousTitForTat	98.6%	1.2%	1.2%	97.4%
TwoTitsForTat	98.5%	1.0%	2.2%	96.3%
Joss	98.4%	1.2%	11.0%	87.4%
TitForTwoTats	50%	49.8%	0.7%	49.3%
DiscriminatingAltruist	1.5%	0.0%	1.5%	0.0%
Bully	1.4%	97.4%	0.3%	1.2%

Analysing the results, we can see that DiscriminatingAltruist and Bully are the only strategies that do not lead to cooperation. From this, we can deduce that in order for cooperation to arise, the agent must be able to react both to defection and cooperation. These two fails as they tend to not be "forgiving" and "retaliating" enough. Here again, the agent converges to defection.

When it comes for TitForTat and its variants, we can see that the Q-learning agent manages to cooperate with them. The first interesting case is against TitForTwoTats : the agent manages to cooperate with it, but only 50% of the time. This is because TitForTwoTats is more forgiving than the others, we can see through the results that the agent understood its opponent strategy and exploited it by defecting as much as possible but not twice in a row (50% of cooperation while getting victimised 0.7% of the time).

Finally, we got the the strategies that applies the "Eye for an eye" rule. All those strategies lead to cooperation from the agent. This is conclusive with the theory, as it is intuitive that the agent should cooperate with strategies that cooperate with it, and defect it when it defects them (on the long run, you prefer to get 3 points every turn rather than 1).

Notably, we see that overpunishing strategies (or just less forgiving strategies) such as Joss and TwoTitsForTat lead to less cooperation. The agent tends to cooperate marginally less but get victimised more often. The Q-learning agent adapts its q-table depending on the it's own actions and the opponent's reactions. Therefore, once it's politics has converged, Joss can easily exploit it as it tends to defect marginally, which is not going to be punished by the agent.

We deduce that the Q-learning agent is able to learn the optimal strategy and that cooperation can arise and be maximised with strategies following conditions : **able to react both to defection and cooperation, "forgiving" and "retaliating" enough, and not overpunishing**.

5 Hyperparameter Sensitivity Analysis

The Q-learning agent updates its Q-table after each round via the Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

5.1 Impact of ε, exploration parameter

In a Q-learning framework, ε controls the trade-off between exploration and exploitation. At each decision that the agent has to make, it will choose to explore with probability ε and exploit with probability $1 - \varepsilon$. Therefore, a high ε means that the agent will try "new things", while a low ε means that the agent will stick to what it knows. In this framework where we want to know if the agent is able to learn the optimal strategy, we need to make sure that the agent has enough time to explore the state space, while exploiting the best strategies it has found at a late stage. Therefore, we choose to start with a high ε and decrease it over time.

Since the agent does not know anything at first, we need to start at **100%** ($\varepsilon_0 = 0.0$ would be a random agent). For the lower bound, we want it to be small enough to ensure that the agent exploits the best strategies it has found at a late stage, while not being 0. The reason why is that it would get victimised too much against opponents like Joss who exploits static strategies. Lastly, we need to choose a decay rate. If it is too high, the agent might converge to a suboptimal strategy. If it is too low, the agent will still be exploring at the end of the game, which is not optimal. This parameter is related to the number of rounds (T=500,000) as we want it to be small after a certain number of rounds.

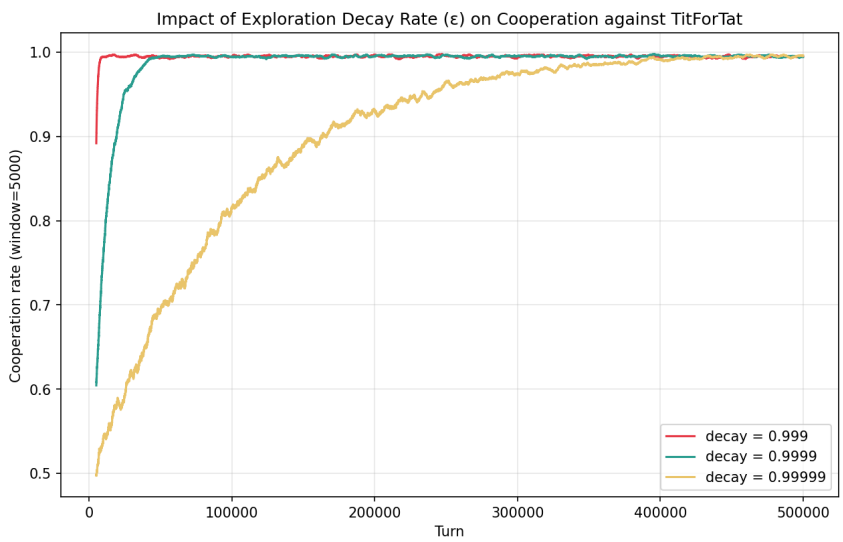

Figure 1: Slower decay allows for a longer exploration phase before locking into cooperation.

For our simulation, we choose $\varepsilon_0 = 1.0$, $\varepsilon_{\text{min}} = 0.01$ and $\varepsilon_{\text{decay}} = 0.9999$.

5.2 Impact of α, learning rate

In Q-learning, α represents the learning rate, which controls how much newly acquired information overrides older information. An α of 0 would mean the agent learns nothing from its environment, whereas an α of 1 would make the agent ignore all past experiences and only consider the most recent reward. Therefore, a high α allows the agent to update its beliefs very quickly, which can be useful in dynamic environments, while a low α ensures stable convergence but makes the learning process significantly slower.

In the context of the Prisoner's Dilemma, setting an appropriate learning rate is important. If α is too high, the agent's Q-values will be overly sensitive to noise or stochastic opponent behaviors, leading to oscillating policies and preventing convergence. If α is too low, the agent will require an unrealistic number of rounds to reach the optimal strategy, exploring poorly before settling down. Thus, a moderate value strikes the right balance between rapid adaptation and Q-value stability.

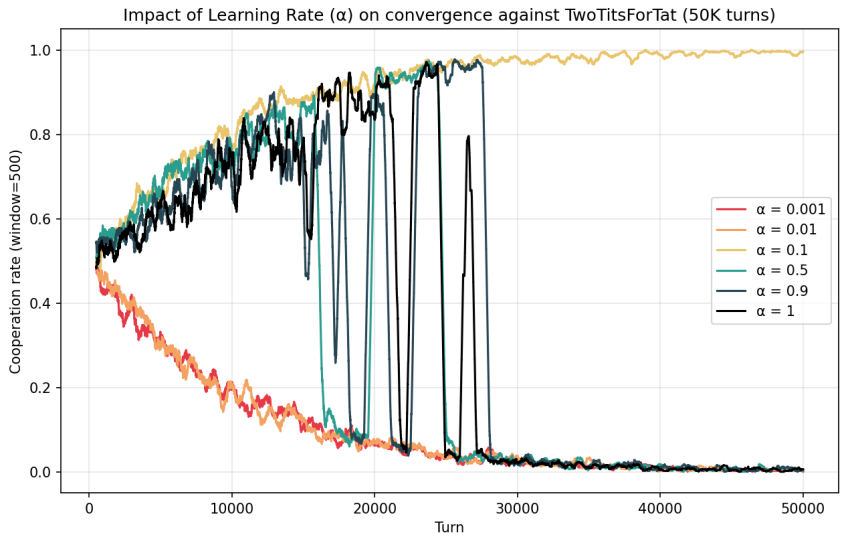

Figure 2: Impact of the learning rate (α) on the convergence against `TwoTitsForTat`.

Againsts TwoTitsForTat, which is a more punitive version of TitForTat, the agent converges to a lower cooperation rate with extreme values of α. Against this strategy, cooperation is the optimal strategy, yet the agent struggles to converge to it with extreme values of α. If α is too high, the agent will tend to forget the past quickly, and since TwoTitsForTat is punishing with a two turns delay, the agent will not be able to learn the optimal strategy (or be highly volatile as we see). Since the exploration decay is at play at the same time, the odds of converging to a suboptimal strategy not negligible. The same way, if α is too low, the exploration rate being high at the beginning, the agent will not learn the optimal strategy against a highly punitive strategy before the exploration rate drops too low (as he will get punished a lot at the beginning).

For our simulation, we choose a standard value of $\alpha = 0.1$, ensuring reliable convergence across all tested adversaries.

5.3 Impact of γ, discount factor

6 Experimental Results

7 Conclusion

References

Robert Axelrod. *The Evolution of Cooperation*. Basic Books, New York, 1984.

Steven Kuhn. Prisoner's dilemma. *The Stanford Encyclopedia of Philosophy*, 2019. URL https://plato.stanford.edu/entries/prisoner-dilemma/strategy-table.html. Accessed: March 2026.

Albert W. Tucker. A two-person dilemma. Mimeograph, Stanford University, 1950.